CampCost — Information Security Policy

Owner: Mason Scarby (Founder / Security Contact) — security@campcost.net

Last reviewed: 2026-07-21

Review cadence: Quarterly, or after any material change to data handling or infrastructure.

CampCost is a personal-finance application that connects to users' bank accounts via Plaid to help them budget trips. Because it processes consumer financial data, security is a first-class requirement, not an afterthought. This policy documents the controls that are **implemented in the product today** and the practices that govern them.

1. Scope

Applies to the CampCost application (React frontend, ASP.NET Core 8 API), its data stores (Supabase Postgres), all third-party integrations (Plaid, Supabase, Stripe, Railway, Cloudflare), and the founder's workstation used to develop and operate it.

2. Access Control

- **Least privilege.** Production systems (Supabase, Railway, Cloudflare, Stripe) are accessed only through the dedicated operator account, separate from personal accounts.
- **MFA is enforced** on all administrative dashboards that store or reach consumer data (Supabase, Railway, GitHub, Cloudflare).
- **Row-Level Security (RLS)** in Postgres restricts every user to their own rows. Sensitive columns on `plaid_connections` (`access_token`, `item_id`, `sync_cursor`) are revoked from client roles entirely — the browser can never read them (migration `003_lock_down_plaid_connections.sql`).
- **API authorization** is enforced server-side on every request via a Supabase-issued RS256 JWT, with issuer pinned to `{Supabase:Url}/auth/v1` and audience `authenticated`. User identity is taken from the verified token, never from the request body.

3. Consumer Authentication (MFA)

- CampCost offers **TOTP-based multi-factor authentication** (Supabase Auth MFA).
- MFA is **required before a user can connect a bank account**: the client gates Plaid Link behind a verified TOTP factor + an `aal2` session, and the API independently rejects bank-token exchange unless the JWT carries `aal == aal2` (403 otherwise).

4. Data Protection

- **In transit:** all traffic is served over TLS 1.2+ (Railway-terminated; the frontend and API are HTTPS-only in production).
- **At rest:** Plaid `access_token` values are encrypted with **AES-256-GCM** (`AesGcmTokenProtector`) before storage; the 256-bit key is held in configuration/secrets, never in source. Supabase Postgres additionally encrypts data at rest at the disk level.
- **Secrets** are never committed to source control; they are injected via `dotnet user-secrets` locally and environment variables in production.
- **No PII, access tokens, or transaction data are written to logs.**

5. Webhook & Integration Security

- Every Plaid webhook must present a valid `Plaid-Verification` ES256 JWT; the signing key is fetched by `kid`, and signature, `iat` freshness, and the `request_body_sha256` body hash are all verified. Unverified requests are rejected (401).
- Unknown `item_id` webhooks are logged and ignored; there is no "sync everything" fallback path.

6. Rate Limiting & Abuse Prevention

- `/api/plaid/*` endpoints are limited to 10 requests/min per user; the webhook endpoint to 30 requests/min per IP. Excess returns HTTP 429.
- CORS is restricted to the production frontend origin via configuration (`Cors:AllowedOrigins`).

7. Vulnerability Management

- **Automated dependency (software-composition) scanning** runs in CI on every push and pull request, and on a **weekly schedule**, via `.github/workflows/vuln-scan.yml`: `dotnet list package --vulnerable --include-transitive` for the API and `npm audit --audit-level=high` for the frontend. A build fails if a High/Critical vulnerability is found, blocking merge until it is resolved.
- **Patch SLA.** Once a vulnerability is identified, it is remediated within a defined timeframe based on severity:

Severity	Remediation SLA

| Critical | 7 days |

| High | 14 days |

| Medium | 30 days |

| Low | Best effort / next release |

- **End-of-life software** is avoided by tracking runtime versions (.NET LTS, current Node LTS) and upgrading before end-of-support.
- Security-relevant fixes are tracked in `CLAUDE.md` (Security & Correctness Issues table) and remediated before go-live.
- **Scope note:** vulnerability management currently covers application dependencies and runtimes. Host/endpoint (laptop, server-instance) scanning is not yet performed; managed hosting (Railway) patches the underlying infrastructure.

8. Change Management

- All changes go through Git; production deploys are from the `main` branch.
- Security-affecting changes are reviewed by the founder before merge.

9. Incident Response

- Suspected incidents are reported to security@campcost.net.
- On confirmed compromise: rotate affected credentials (Plaid, Supabase, Stripe keys), assess scope of exposed data, and notify affected users and Plaid as required by applicable law and the Plaid developer agreement.

10. Data Privacy & Retention

- Governed by the published Privacy Policy and the Data Retention & Deletion Policy.

11. Review

This policy is reviewed at least quarterly and updated whenever data handling, infrastructure, or third-party integrations change materially. Review date and owner are recorded at the top of this document.